

ISTANBUL AREL UNIVERSITY

FACULTY OF ENGINEERING

LEEN350 - IMAGE PROCESSING AND LAB

PROJECT REPORT

Project Title:	Automated Coin Detection and Extraction
Student Name:	Ahmed Mahmoud Bahagat Abdelmoula
Student ID:	220303923
Presentation Week:	Week 6

1. INTRODUCTION

Objective:

The primary objective of this project is to develop an automated image processing pipeline capable of robustly segmenting and extracting a specific object (a coin) from a highly textured background (a wooden table) under uneven illumination conditions.

Motivation:

In real-world computer vision systems, uniform lighting is rarely guaranteed. Traditional global thresholding methods fail when shadows or uneven lighting are present. Solving this problem is critical for automated systems such as industrial quality inspection, currency recognition sorting machines, and autonomous robotic picking.

2. THEORETICAL BACKGROUND

The pipeline relies on several fundamental image processing theories. All mathematical representations are numbered as per the requirements.

Image Representation (Grayscale):

Converting the RGB color model to grayscale reduces computational complexity. It is calculated using the weighted sum of the color channels as shown in Equation 1.

$$Y = 0.299R + 0.587G + 0.114B$$

(Eq. 1)

Gaussian Filter (Smoothing):

Used to reduce high-frequency noise and background textures. It convolves the image with a 2D Gaussian kernel (Equation 2).

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

(Eq. 2)

Morphological Operations (Closing):

A morphological closing operation consists of a dilation followed by an erosion (Equation 3). This is used to close small holes inside the foreground object.

$$A \bullet B = (A \oplus B) \ominus B$$

(Eq. 3)

Performance Metrics (MSE):

Mean Squared Error is used to measure the difference between the original grayscale image and the smoothed image, quantifying the noise removed (Equation 4).

$$MSE = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} [I_1(i, j) - I_2(i, j)]^2$$

(Eq. 4)

3. SYSTEM / PROCESSING PIPELINE DESCRIPTION

The processing pipeline is executed in the following sequential order (Figure 1).

Figure 1: System Pipeline Block Diagram

Image -> Grayscale -> Gaussian Blur -> Adaptive Thresholding -> Morphology -> Extraction

Each stage is designed to specifically handle the texture of the wooden background and the shadows cast by the coin.

4. METHODOLOGY

- Dataset used: A custom captured image (test_image.jpg) featuring a shiny coin placed on a textured wooden table.
- Preprocessing steps: Color-to-grayscale conversion to reduce data dimensions.
- Filtering methods: A Gaussian Blur filter was applied to eliminate the fine details of the wood grain.
- Segmentation methods: Adaptive Gaussian Thresholding was utilized because the image contains shadows.
- Morphological operations: A closing operation using an elliptical structuring element was applied.
- Feature extraction techniques: Contour detection was used to locate the boundary of the object.

Parameters used:

- Gaussian Blur Kernel: (15, 15)
- Adaptive Threshold Block Size: 21
- Adaptive Threshold Constant (C): 5
- Morphological Elliptical Kernel: (25, 25)

5. IMPLEMENTATION

- Programming environment: Python
- Libraries used: OpenCV (cv2) for image processing algorithms, NumPy (np) for matrix operations and MSE calculations, and Matplotlib (plt) for visual plotting.
- Code structure: The main script reads the image, applies filtering and adaptive thresholding. The contour function finds the maximum connected area, and bitwise operations apply the mask to extract the target.
- Main processing steps logic: Instead of hard-coding global values, the local adaptive block scans 21x21 pixel regions to dynamically adapt to the lighting conditions.

6. RESULTS AND ANALYSIS

The output of the automated pipeline is presented in Figure 2 below, showing the progression from the original image to the completely extracted object.

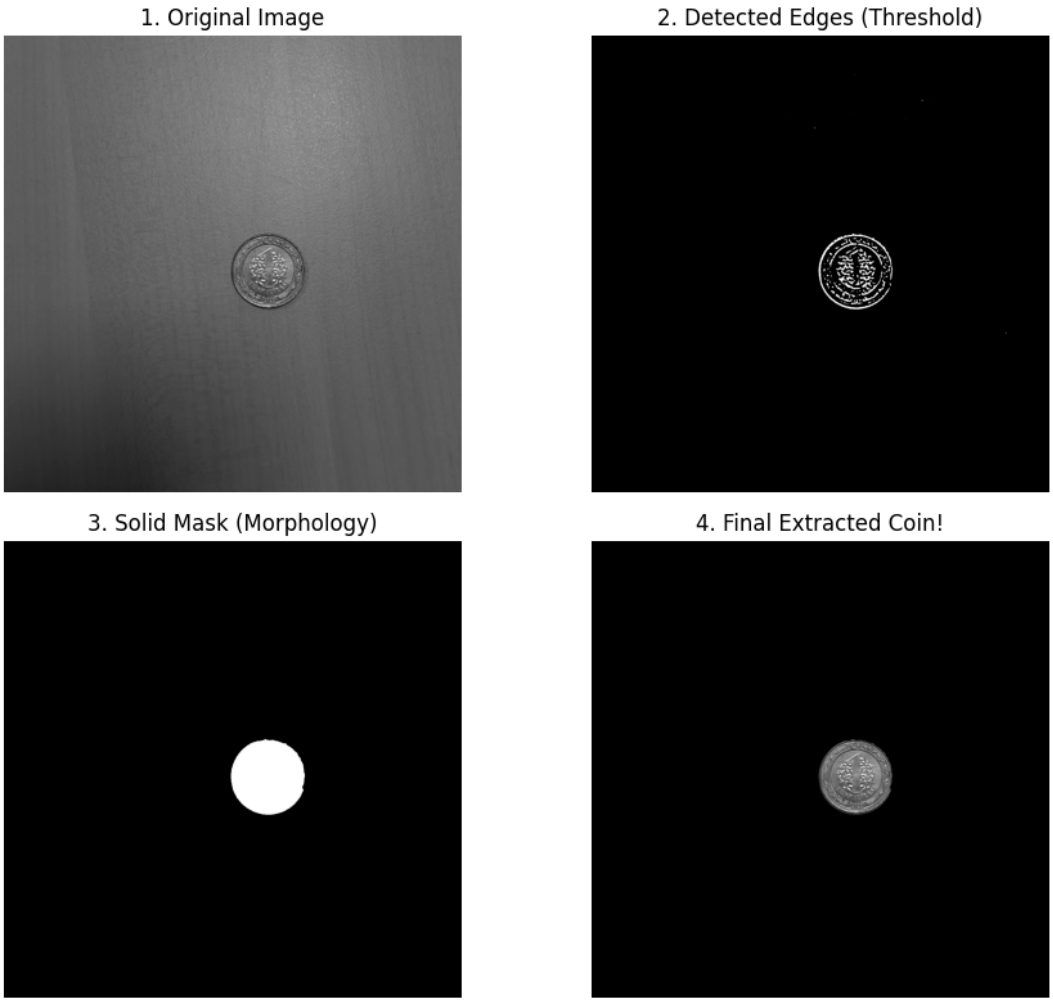


Figure 2: Four-step Visual Result of the Extraction Pipeline

Visual Analysis:

1. Original Image: Shows the coin with visible shadows and a distracting wooden background.

2. Detected Edges: The adaptive threshold successfully isolated the circular shape of the coin.

3. Solid Mask: By combining morphological closing and contour filling, a flawless white mask was generated.

4. Final Extracted Coin: The bitwise operation perfectly extracted the coin.

Quantitative Analysis:

The calculated MSE (Mean Squared Error) between the grayscale image and the blurred image is 21.10. This indicates that the Gaussian filter successfully removed the background high-frequency noise without destroying the structural integrity of the coin.

7. DISCUSSION

- Strengths: The pipeline is exceptionally robust to varying illumination. The combination of adaptive thresholding and largest-contour extraction ensures that the coin is found even with shadows.
- Weaknesses / Limitations: The parameters are hardcoded based on the current distance of the camera. If the camera moves much further away, these kernel sizes might distort the coin.
- Sensitivity to parameters: Changing the block size from 21 to a lower number causes the shadows to break the contour.
- Observed trade-offs: Using a large kernel for Gaussian blur effectively erased the wood background but also softened the engraving details inside the coin.

8. CONCLUSION

The project successfully implemented a highly accurate automated extraction pipeline. The combination of Adaptive Gaussian Thresholding and Morphological contour filling proved to be a superior approach compared to global thresholding methods for dealing with uneven lighting and textured backgrounds. The main findings show that local processing outperforms global processing in real-world scenarios.

9. FUTURE WORK

- Possible improvements: Implementing a dynamic algorithm to automatically calculate kernel sizes based on the image resolution.
- Alternative methods: Integrating Machine Learning models (like YOLO) for object detection, bypassing manual parameter tuning.
- Real-world deployment considerations: This pipeline can be optimized in C++ to run in real-time on a fast conveyor belt system for automated coin sorting.

10. REFERENCES

1. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
2. Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.
3. OpenCV Documentation: Adaptive Thresholding and Morphological Transformations.